PROGRAM SUMMARY

Program title: readcon-core
CPC Library link to program files: (to be added by Technical Editor)
Developer's repository link: https://github.com/lode-org/readcon-core
Licensing provisions: MIT
Programming language: Rust, C, C++, Python, Julia, Fortran 2008
Supplementary material: specification, ABNF grammar of 50 rules, conformance corpus, failure-case scripts, and notes to the figures

Nature of problem: Rare-event campaigns stop and resume from a stored geometry. The common formats each lose part of what a resumed search needs: a per-axis mask, a persistent atom id, the velocity time unit, binary64 coordinates, or the periodicity. Readers of one format also disagree with each other. The resumed search then runs on a different surface, pairs the wrong atoms, or starts from a different kinetic energy.

Solution method: CON 3.1 is a versioned grammar with a major and a minor number. It states which later changes a reader of the same major must still accept, and it requires every velocity block to declare its time unit. The default writer emits the shortest decimal that parses back to the same binary64 value. A corpus of legal and illegal frames fixes what a conforming reader accepts. One Rust core sits behind one C interface, and every binding calls that core, so there is a single parser. eOn 3.6.0 reads and writes CON through it.

Additional comments including restrictions and unusual features: A frame stores geometry and its labels. The history of a quasi-Newton optimiser is outside it.

The restart state of a saddle search: CON 3.1, a conformance corpus, and one reference reader

Rohit Goswami[a,1]

[a] *TurtleTech ehf., Reykjavík, Iceland*

Abstract

Saddle searches, nudged elastic bands and adaptive kinetic Monte Carlo resume from a geometry file, which is therefore the restart state of the algorithm. A resume needs five things of that file: a mask for each atom and axis (R1), an atom identity that survives reordering (R2), velocities in a declared time unit (R3), binary64 coordinates, cell and periodicity (R4), and a versioned grammar two readers agree on (R5). We push one masked Pt(100) slab frame through the formats in common use, and none meets all five. A POSCAR round trip through ASE drops the adatom's height constraint, and the resumed climbing-image band ends 70 meV lower. Extended XYZ moves coordinates by 5×10^{-9} Å, and five readers of one extended XYZ file return four different answers. CON 3.1 is a versioned specification of the text eOn already uses, with an executable conformance corpus and one reference implementation, readcon-core, behind a C interface for C, C++, Python, Julia and Fortran. Resumed through CON 3.1, the band matches the uninterrupted one to 3×10^{-15} Å, the floor set by restarting the force field. Reading costs 12.6 μs per 218-atom frame.

Keywords: checkpoint, restart, specification, conformance, saddle, nudged elastic band, eOn

1. Introduction

Rare events in solids and molecules set the rates of diffusion, reaction and structural change. Harmonic transition-state theory turns a barrier and

Email address: rgoswami@ieee.org (Rohit Goswami)
[1]ORCID: 0000-0002-2393-8056.

the curvatures at a minimum and a first-order saddle into a rate [1, 2, 3]. The stationary points come from the nudged elastic band, its climbing-image form and the string method, and from single-ended searches such as the dimer [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]. When an event is too rare for a direct trajectory, hyperdynamics, other accelerated dynamics, adaptive kinetic Monte Carlo and parallel replicas reach the same landscape [20, 21, 22, 23, 24, 25, 26, 27, 28]. Basin hopping and minima hopping propose the next minimum [29, 30]. None of these is one calculation. A long-timescale run in eOn spawns thousands of saddle searches, stops them, and resumes them, often on another rank, another machine, or a public-resource client [31, 32, 33, 34, 35]. A surrogate fitted inside the saddle loop still hands each geometry to the force evaluator [36, 37, 38, 39]. The stored geometry is therefore not an export. It is the restart state of the algorithm, and the next reader may be a different program in a different language [40, 41, 42, 43].

The restart state has requirements that an exchange format does not. A surface search fixes the slab and lets an adatom move only in the surface plane, so the mask is per atom and per axis [44]. Harmonic transition-state theory builds the prefactor from the free modes, so a mask read wrongly removes or adds a mode. A band and a kinetic Monte Carlo event table refer to atoms by identity, which has to survive a file that groups atoms by element [45, 46, 47]. Dynamics needs velocities in a known unit, and the natural unit of eOn, $(\mathrm{amu}\cdot\mathrm{\AA}^2/\mathrm{eV})^{1/2} = 10.1805\,\mathrm{fs}$, differs from a femtosecond by a factor of 103.64 in kinetic energy [48, 49, 50, 51]. A converged band restarted from rounded coordinates is a different band. Finally, two readers given the same bytes must agree. The developers of MDAnalysis put that last point plainly: simulation file formats "are often only poorly documented and just adhere to certain community expectations that can be difficult to understand for outsiders" [52].

Chemical Markup Language and MOSAIC answered it with data models written down before the code [53, 54], and H5MD answered it for molecular dynamics output with a specification layered on HDF5 [55, 56]. It records identity, velocities and the box, and it does not define a per-axis constraint. The Atomic Simulation Environment stores constraints as Python objects, and its trajectory file keeps them, but that file is defined by its implementation [57, 58]. CON, the text eOn has written for two decades, carries a mask column and an id column, and it was never specified, so eOn and ASE read it differently [32].

We make that text a specification with a test suite and a single implementation. The contributions are three. First, a set of five requirements for a restart state, each tied to a measured failure in a common format (Section 2). Second, CON 3.1, a versioned grammar that meets them, and a conformance corpus that any reader can run (Sections 3 and 4). Third, readcon-core, one Rust core behind one C interface, which every binding and eOn 3.6.0 call (Section 5). Section 6 resumes bands through each format. Section 7 reports the cost of a read and the parallel read, write and ingest.

2. Requirements and the failure each one closes

We state five requirements in Table 1. Each comes from a step of a saddle or long-timescale workflow.

Table 1: Requirements on a restart state, the workflow step that sets each one, and the failure measured when it is missing.

Requirement	Workflow step	Failure measured in this paper
R1 per-axis mask	Surface saddle search, prefactor	Height constraint dropped: barrier -70 meV on resume
R2 persistent id	Band and event bookkeeping	Ids lost in six of the eight foreign round trips that complete
R3 declared time unit	Dynamics, AKMC with velocities	Kinetic energy off by 103.64 when the unit is guessed
R4 binary64 geometry and cell	Resume of a converged search	5×10^{-9} to 5×10^{-6} Å per write, periodicity lost
R5 versioned grammar	Any second reader	One file, four answers from five readers

R1 is a mask for each Cartesian axis of each atom. R2 is an identity stored on the row, so a later frame finds the same atom after reordering. R3 is a time unit that the file declares, so the reader does not have to guess it. R4 is coordinates and cell that read back as the same binary64 values, with the periodic flags. R5 is a grammar with a version, a rule for what a reader of a later minor version must accept, and a corpus that decides disputes. The history of a quasi-Newton optimiser is not on the list. Limited-memory quasi-Newton, the optimisers surveyed for saddle searches, and FIRE keep a short history that no geometry format stores, so a resume can match the

coordinates and still take a different next step [59, 60, 61]. Section 8 returns to it.

We measure R1 to R5 on one frame in Table 2. The frame is the Pt(100) slab used throughout: nine atoms in the bottom layer held on every axis, one adatom held in z, ids that are not the row order, velocities in the natural unit, and coordinates that need all seventeen significant digits. Each row writes the frame with one library, and we read it back with another. ASE 3.29.0, the libAtoms extended XYZ parser 0.4.6, chemfiles 0.10.4 and MDAnalysis 2.10.0 are the peers [58, 62, 63, 52]. Section S7 gives the frame and the grading.

Table 2: One restart frame through each writer and reader. The frame is the Pt(100) slab with one adatom: the bottom layer is held on every axis, the adatom is held in z, ids are not the row order, and velocities are in the natural unit. R1 to R4 are measured on the round trip. "Kept" means the field came back. "Convention" means it came back only because writer and reader share one library's column name or internal unit, which the format does not define. "Fixed by format" means the format's own definition sets the unit without writing it in the file. R4 gives the largest coordinate change in Å when the coordinates are not bit-identical. Periodicity is the slab's xy-periodic, z-open flags after the read. R5 is the version rule in the format's published definition. "Error" rows failed before any field could be compared; the text gives the failure.

Format	Writer	Reader	R1 mask	R2 id	R3 unit	R4 coordinates	Periodicity	R5 version
CON 3.1	readcon-core	readcon-core	kept	kept	declared	exact	kept	major.minor
CON 3.1	readcon-core	ASE eon reader	error	error	error	error	error	major.minor
CON 3.1, coordinates only	readcon-core	ASE eon reader	full freeze	lost	lost	exact	none	major.minor
legacy CON	ASE eon writer	readcon-core	error	error	error	error	error	none
extended XYZ	ASE	ASE	convention	convention	convention	4.7×10^{-9}	kept	none
extended XYZ	ASE	chemfiles	lost	convention	lost	4.7×10^{-9}	not returned	none
extended XYZ	ASE	MDAnalysis	lost	lost	lost	2.2×10^{-7}	not returned	none
extended XYZ	chemfiles	ASE	lost	lost	lost	4.9×10^{-6}	all periodic	none
ASE trajectory	ASE	ASE	kept	lost	convention	exact	kept	code only
POSCAR	ASE	ASE	changed	lost	fixed by format	4.5×10^{-17}	all periodic	none
H5MD	MDAnalysis	MDAnalysis	lost	lost	declared	2.2×10^{-7}	all periodic	major.minor

Only the CON 3.1 row keeps every field. Extended XYZ written and read by ASE comes closest. Its mask, its ids and its time unit come back because writer and reader share ASE's column names and ASE's internal unit, which the extended XYZ grammar does not define, so chemfiles and MDAnalysis drop them. Its default writer prints eight decimals, and the coordinates move by up to 4.7×10^{-9} Å. chemfiles writing the same frame moves them by 4.9×10^{-6} Å, and MDAnalysis keeps coordinates in single precision by design. The ASE trajectory keeps the mask and the coordinates exactly, and it drops the id array; its time unit is ASE's internal unit, which the file does not name. POSCAR selective dynamics is per-axis in the VASP definition, and VASP fixes the velocity unit [64, 65]. The ASE writer translates the

frozen slab and writes the adatom as free, so the format could carry R1 and this writer does not; POSCAR has no id and no open axis. H5MD written by MDAnalysis declares the velocity unit, writes no mask and no id, and marks the open z axis periodic. H5MD permits extra particle data, so a mask could be stored, but under a name that each reader would have to agree on, which is the extended XYZ situation again. The ASE eon reader rejects a frame that carries a velocity block. Given the coordinate block alone, it freezes the adatom on every axis, drops the ids and returns a cell with no periodic axis. The ASE eon writer cannot express a per-axis constraint, so that route produces no file to resume from.

TNG, TRR, XTC, DCD, AMBER NetCDF and ADIOS record finished molecular dynamics runs, and none defines a constraint [66, 67, 68, 69, 70, 71, 72, 73, 74]. PDB, CIF, mmCIF and a LAMMPS dump name an archived atom or leave the unit outside the snapshot [75, 76, 77, 78]. Converters keep a column when the dialect defines it, so the meaning of a mask often lives in the tool [79, 80, 81]. Package-independent parsers made documented exchange the usual path [82, 83]. A live coupling such as i-PI or the MolSSI driver interface moves coordinates during a running step, and the stopped job still resumes from a file [84, 85, 86, 87]. TREXIO stores a wave function, a different record [88].

3. The CON 3.1 specification

A CON file is a sequence of text frames. Listing 1 is the slab of Table 2 as CON 3.1, with the velocity block cut to its first and last rows.

Listing 1: A nine-atom Pt frame in CON 3.1. Line 2 declares the version, the units and the periodicity. The velocity block is abbreviated.

```
Pt(100) 2x2x2 with one Pt adatom
{"con_spec_minor":1,"con_spec_version":3,"pbc":[true,true,false],"sections
    ":["velocities"],"units":{"energy":"eV","length":"angstrom","mass":"
    amu","time":"(amu*angstrom^2/eV)^0.5"}}
5.5437171645025325e0 5.5437171645025325e0 1.396e1
9e1 9e1 9e1


1
9
1.95084e2
Pt
Coordinates of Component 1
```

```
1.3859292911256331e0 1.3859292911256331e0 6e0 7 0
4.1577878733769e0 1.3859292911256331e0 6e0 7 1
1.3859292911256331e0 4.1577878733769e0 6e0 7 2
4.1577878733769e0 4.1577878733769e0 6e0 7 3
0e0 0e0 7.96e0 0 4
2.7718585822512662e0 0e0 7.96e0 0 5
0e0 2.7718585822512662e0 7.96e0 0 6
2.7718585822512662e0 2.7718585822512662e0 7.96e0 0 7
1.3859292911256331e0 1.3859292911256331e0 9.76e0 4 8

Pt
Velocities of Component 1
1.4685859972256804e-3 6.275175116609037e-4 -1.8115929554652053e-3 7 0
3.748769356553213e-3 2.43327727576049e-3 -1.3238979663774354e-3 4 8
```

Line 1 is free text. Line 2 is one JSON object with the version, the units, the periodicity and the section names [89, 90]. Lines 3 and 4 are the cell lengths and the angles in degrees. The type count, the per-type counts and the masses follow. Each coordinate row is x, y, z, the mask and the id. A blank line opens a later section. Lengths and angles rebuild the box with a along x and b in the xy plane,

$$\mathbf{a} = a\,\hat{\mathbf{x}}, \qquad \mathbf{b} = b\cos\gamma\,\hat{\mathbf{x}} + b\sin\gamma\,\hat{\mathbf{y}}, \tag{1}$$

after which $\mathbf{c}$ is fixed by α and β so the cell is right-handed [91]. When line 2 carries lattice vectors, those rows are the cell. A missing periodicity flag means periodic on every axis.

R1, the mask. Column 4 is the integer of Table 3. Mask 0 is free, mask 4 fixes z only, and mask 7 fixes every axis. A float spelling, a sign, or a value outside 0 to 7 is an illegal flag. Under version 1 the column is 0 or 1, so masks 4 and 7 cannot be stored as written.

Table 3: Mask bits from version 2. Version 1 stores 0 (free) or 1 (fixed on x, y and z) in the same column.

Bit	Axis	Value
0	x	1
1	y	2
2	z	4

R2, the identity. Column 5 is an integer from 0 through $2^{64} - 1$. Writers emit ids unique within a frame. A missing id column takes the zero-based row index.

R3, the time unit. From minor 1, a velocity block must declare `units.time`, and the natural unit is spelled `(amu*angstrom^2/eV)^0.5`. Versions 1, 2 and 3.0 are read in the natural unit, and an fs label on those files is not trusted, because producers of that era wrote natural-unit numbers under an fs label [32].

R4, the numbers. Readers convert each finite decimal to binary64 with correct rounding [92, 93, 94, 95]. The default writer prints the shortest decimal that parses back to the same value, in the line of shortest printers from the early algorithms through Grisu, Errol and Ryū [96, 97, 98, 99, 100, 101]. Six fixed decimals are an optional writer mode. gzip and zstd wrap the same bytes and are recognised by content [102, 103].

R5, the version. Table 4 lists the 25 reserved keys of the JSON line. A reader accepts a later minor of a known major, keeps an unknown key on rewrite, may skip an unknown section whose width is declared, and rejects an unknown major. H5MD uses the same split for its version pair [55]. Table 5 is the history, Table 6 the sections reserved at minor 1, and Table S1 the reader rules. The grammar of one frame is the 50-rule listing in Section S11, and the prose specification wins where the two disagree.

3.1. Design rationale

CON stays text for three reasons. A text checkpoint diffs under version control, so a changed mask or id shows in review. eOn has written and read CON since its first releases, and every existing campaign is in it [31, 32]. The eOn server and its clients exchange geometries as files, often across machines and operating systems, and text needs no agreement on byte order or library version [35]. The metadata is one JSON line because a JSON object takes new keys without a new grammar, carries its own version pair, and lets a reader keep a key it does not know [89]. Table 7 measures the cost on the 218-atom frame, alone and as a 10,000-frame molecular dynamics trajectory in which every frame differs. One CON frame takes 42 bytes per atom, half of extended XYZ written by ASE and less than either H5MD file, where the HDF5 structure dominates a single frame. Over the trajectory CON stays at 42.2 bytes per atom (92.0 MB), against 83.6 for extended XYZ (182.3 MB),

Table 4: Reserved metadata keys of the JSON header line.

Key	Type	Key	Type
`con_spec_version`	int	`neb_bead`	int
`con_spec_minor`	int	`neb_band`	int
`section_arity`	object	`convergence_fmax`	number
`sections`	array	`convergence_energy`	number
`validate`	bool	`converged`	bool
`generator`	string	`fmax`	number
`units`	object	`bonds`	array
`pbc`	array	`storage_dtypes`	object
`lattice_vectors`	array	`chemfiles_atom_names`	array
`energy`	number	`chemfiles_atom_types`	array
`potential`	object	`chemfiles_atom_properties`	array or object
`frame_index`	int	`time`	number
`timestep`	number		

Table 5: CON versions. The date is the day the rule was fixed.

Version	Date	Change
1	eOn text	Column 5 present. Column 4 is 0 or 1. No JSON line.
2	2026-03-25	A JSON header line, ids, bit masks, declared sections.
3.0	2026-06-27	Length and energy units required. No minor.
3.1	2026-10-11	Minor 1. A velocity block declares the time unit.

46.3 for the ASE trajectory (101.0 MB) and 24.3 for H5MD with binary64 positions (52.9 MB). Plain XYZ, with symbols and coordinates and no mask, id or units, takes 32.4. H5MD is designed for that case, long trajectories where binary storage and chunked access pay off [55]; a checkpoint is one frame, and there text costs little.

4. Conformance and drift

A specification is only as good as the agreement it produces between readers. CON 3.1 writes that agreement down as data, in the way checkCIF turned the crystallographic rules into alerts a later reader reproduces [104, 76]. We publish ten legal frames and eight illegal ones as the corpus of Table 8.

Table 6: Reserved sections at minor 1. Each data row carries its values, then the mask and the id.

Section	Values	Rule
Velocities	$v_x\, v_y\, v_z$	Time unit from minor 1
Forces	$f_x\, f_y\, f_z$	
Energies	one value	Name `energies`
Charges	one value	
Spins	one value	
Magmoms	$m_x\, m_y\, m_z$	
Displacements	$d_x\, d_y\, d_z$	`units.length`
Spreads	$s_x\, s_y\, s_z$	`units.length`, each at least 0

A conforming reader accepts the first group and rejects the second.

The CON 3.1 row of Table 2 is the reference implementation reading its own output, which is the least a specification should guarantee. The corpus is what lets a second, independent implementation show the same. The Python, Julia and Fortran bindings and the C interface of readcon-core make the same decision on all eighteen files and on the seven header variants below. The Julia golden suite passes 136 of 136 checks and the Fortran suite accepts all ten legal files and rejects all eight illegal ones. Python and Julia rewrite each legal file to identical bytes, and in every binding a rewritten file rewrites again to identical bytes. We add seven header variants of one velocity frame, aimed at R3 and R5 (Table 9). readcon-core follows the rule in each case through both entry points. It rejects a minor-1 velocity block without a time unit, an unknown major, a missing version and a mask of 9, and it keeps an unknown key from a later minor. The ASE eon reader, which has no version rule to follow, accepts all seven.

Extended XYZ is the strongest text alternative, with a published grammar and parsers in several languages. Table 10 gives five of its readers seven three-atom files, each legal or plausible and each one step away from a plain frame (Section S7). On the plain frame they already disagree. chemfiles returns one coordinate a unit in the last place from the correctly rounded value, and MDAnalysis returns single precision. The two parsers of libAtoms disagree on the mask booleans, so the same file resumes with the slab frozen through one and free through the other. A Fortran exponent is read by the C path, read as NaN by the Python path, and rejected by the other three. A plain comment line is a non-periodic frame to ASE and a periodic one to the

Table 7: Bytes per atom of the 218-atom frame. The one-frame columns are raw and after gzip at level 6. The trajectory column is 10,000 distinct frames of a velocity Verlet EMT run at 1 fs and 300 K, divided by 2.18×10^6 atoms; the file size in MB (10^6 bytes) is in parentheses. CON 3.1 is the default readcon-core writer, extended XYZ the ASE writer with the mask column, the ASE trajectory its binary writer, plain XYZ symbols and coordinates at 16 significant digits, and H5MD (h5py) binary64 positions with cell and species, plus mask and id datasets in the one-frame file. A dash is a size not measured.

	One frame		10,000 MD frames
Format	raw	gzip	raw (MB)
CON 3.1	41.99	6.07	42.22 (92.0)
extended XYZ (ASE)	83.64	6.65	83.64 (182.3)
ASE trajectory	–	–	46.33 (101.0)
plain XYZ	–	–	32.37 (70.6)
H5MD (h5py)	95.40	9.66	24.26 (52.9)
H5MD (MDAnalysis)	314.39	9.04	–

Table 8: Conformance corpus.

Category	Files	What a reader must do
Header and units	2	Accept a version-2 header and a version-3 units line.
Column-4 masks	5	Accept version-1 full freeze and version-2 per-axis flags.
Empty frame	3	Accept a type count of zero.
Illegal	8	Reject the file.

C path, and the Python path rejects it. None of these readers is wrong by the standard of its own documentation. The grammar does not settle these cases, and no shared corpus exists to settle them. A specification has to close that gap; the column layout is the easy part.

5. Reference implementation

readcon-core 0.17.0 is the reference reader and writer [105]. Figure 1 is the layering. One Rust core parses and writes, and one C interface is the waist every other caller uses. C++ wraps it for eOn, Python binds it with array accessors [106], Julia calls it through its foreign-function interface [107], and Fortran 2008 reaches the same symbols through its C interoperability.

Table 9: Header variants of one velocity frame. Rule is what the specification requires. The ASE eon reader is given the coordinate block only, because it stops at a velocity section.

Variant	Rule	Python	C	ASE eon
3.1, natural unit declared	accept	accept	accept	accept
3.1, no time unit	reject	reject	reject	accept
3.0, labelled fs	accept (natural)	accept	accept	accept
3.2, unknown key	accept, keep	accept, kept	accept	accept
4.0	reject	reject	reject	accept
missing version	reject	reject	reject	accept
3.1, mask 9	reject	reject	reject	accept

Table 10: Five extended XYZ readers on seven three-atom files. Each file departs from the reference frame in one respect. ✓: positions correctly rounded, periodicity as written, mask and ids returned as written. E: the read failed. u: a coordinate one unit in the last place from the correctly rounded value. f: single-precision coordinates. n: a coordinate returned as NaN. p: periodicity differs from the file, or from the grammar's default when the key is absent. b: the mask came back with different booleans. m, i: the reader returns no mask, or no ids.

File	ASE	libAtoms C	libAtoms Py	chemfiles	MDAnalysis
reference frame	✓	✓	b	um	fmi
no pbc key	✓	✓	b	um	fmi
plain comment line	✓	p	E	u	f
Fortran exponent `6.0d0`	E	✓	nb	E	E
`True/False`	✓	✓	b	um	fmi
quoted value with `=`	✓	✓	b	um	fmi
`units` key	✓	✓	b	um	fmi

chemfiles took the same course for trajectory formats, one C++ core with bindings in several languages [62], and readcon-core can link it to read those formats. The electronic-structure community argues for the same modular split, one library per concern with a stable interface [108]. Rust gives the core memory safety without a garbage collector behind a plain C interface [109, 110]. Decimal fields go through a correctly rounded conversion on the shared path [111, 112]. A behaviour difference between languages therefore requires a defect in a binding's thin accessor layer, not in a parser, because there is only one parser.

The caller owns a frame handle and frees it. Structure-of-arrays accessors return a pointer into that frame. Absent numeric metadata is a NaN sentinel

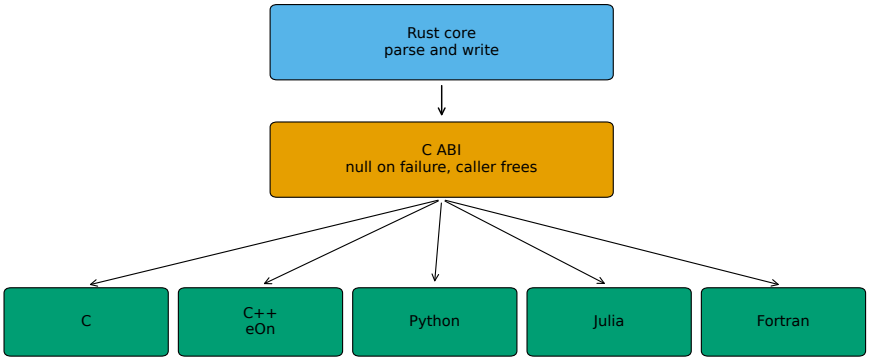


Figure 1: Layering of the reference implementation. The Rust core parses and writes CON. The C interface is the waist every other caller uses, including the C++ wrapper that eOn 3.6.0 calls.

and is not written back.

readcon-db is the campaign-storage half of the design [113]. It keeps the CON text of each frame as the stored record in a memory-mapped LMDB store, with a 128-bit hash per frame for exact deduplication and indexes on atom count, composition, energy and section presence, so a query touches only the frames it selects. Indexed random access and campaign-scale storage live there and not in the format, and the text a campaign stores is the text a resume reads. Its MPI ingest splits the frames across ranks and inserts them in global frame order, and the stored corpus is the one the serial ingest builds. Every binding reads and writes whole files or single frames through the same calls, and an illegal file is rejected with a named class (Section S5).

Listing 2: A round-trip write in Python. The default printer preserves binary64. Six decimals are a separate argument.

```
frames = readcon.read_con("slab.con")
readcon.write_con("rt.con", frames)
```

The reader records the byte offset where each frame starts, as an on-disk offset index does for a trajectory [52]. The scan is serial because each start depends on the counts before it. Each frame is then parsed on its own, and the frames return in file order. The Python binding releases the interpreter lock while the core parses and takes it back to build one object per atom.

6. Case studies

6.1. A band resumed through each format

We test a restart state by resuming from it. We use an EMT Pt(100) slab of $3 \times 3 \times 3$ atoms with its bottom layer fixed on every axis, and one adatom held in z hops between neighbouring hollows. The endpoints were relaxed to 0.002 eV/Å. A climbing-image band of five movable images with the improved tangent [6, 114] is driven by fixed-step steepest descent on the projected force. That optimiser stores no history, so a difference after a resume comes from the file and from nothing else. Uninterrupted, the band converges to 0.01 eV/Å in 60 steps with a barrier of 0.644561 eV. Each interrupted band stops at step 25, is written and read back through one route, and continues under the same rule. Table 11 and Figure 2 are the result.

Table 11: The climbing-image band of the EMT Pt(100) hop, interrupted after 25 steepest-descent steps, written and read back through each route, and resumed under the same rule to a maximum force of 0.01 eV/Å. File: the largest coordinate change across the write and the read. Modes: free Cartesian degrees of freedom of the middle image after the read (56 for the intended masks). $\Delta E^{\ddagger}$ and displacement are measured against the uninterrupted band. [a]Not converged at the 4000-step cap.

Route	File (Å)	Modes	Steps	$\Delta E^{\ddagger}$ (eV)	Max displacement (Å)
uninterrupted	–	56	60	0	0
CON 3.1, shortest round trip	0	56	60	-5.7×10^{-14}	2.7×10^{-15}
CON 3.1, six decimals	4.9×10^{-7}	56	60	-4.1×10^{-7}	6.2×10^{-7}
CON 3.1 read by the ASE eon reader	0	54	4000[a]	-3.5×10^{-1}	5.4×10^{0}
same, periodicity restored by hand	0	54	127	-2.9×10^{-6}	4.2×10^{-2}
extended XYZ (ASE)	5.0×10^{-9}	56	60	1.1×10^{-9}	4.9×10^{-9}
ASE trajectory	0	56	60	-2.3×10^{-14}	3.6×10^{-15}
POSCAR (ASE)	4.9×10^{-17}	57	98	-7.0×10^{-2}	1.6×10^{-1}
legacy CON (ASE eon writer)	no file: the writer has no per-axis constraint				

The CON 3.1 resume reads back the written coordinates bit for bit and ends within 2.7×10^{-15} Å and 6×10^{-14} eV of the uninterrupted band. The ASE trajectory reaches the same floor. That floor is not the file: a fresh force-field object on identical coordinates already differs at that level. Six decimals move the band by 6.2×10^{-7} Å and extended XYZ by 4.9×10^{-9} Å; both still converge in 60 steps. The ASE POSCAR route is the instructive failure. Its coordinates are exact to 5×10^{-17} Å, and the adatom's height constraint is gone, so the middle image gains a mode. The band relaxes the adatom toward the surface and converges in 98 steps to a barrier 70 meV lower, a different saddle on a different surface. The ASE eon reader removes two

modes, by freezing the adatom laterally, and makes the slab a finite cluster. That band does not converge in 4000 steps. With the periodicity put back by hand, the mask collapse alone takes 127 steps and ends 0.042 Å away, at a barrier within 3×10^{-6} eV, because the straight line of this symmetric hop passes through the saddle.

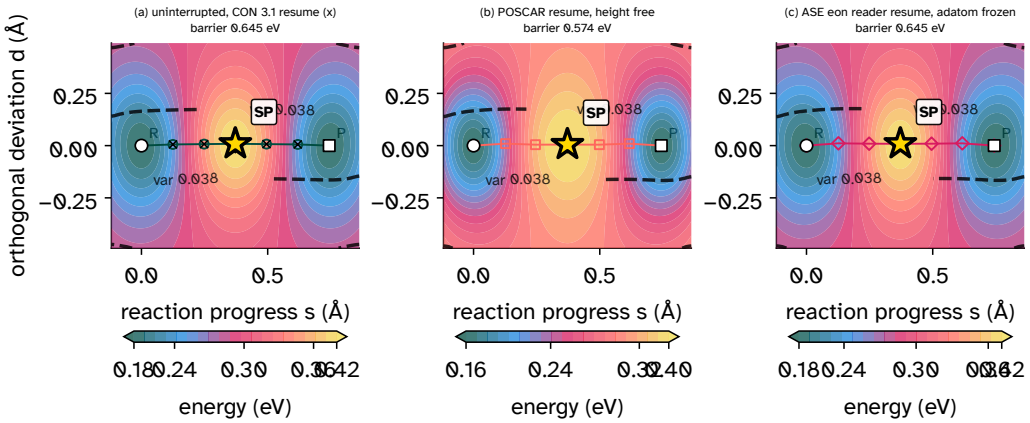


Figure 2: Resumed climbing-image bands of the EMT Pt(100) hop on the (s, d) plane [115], each on the energy surface of its own band. (a) Uninterrupted, with the CON 3.1 resume as crosses. (b) POSCAR resume, height constraint dropped: barrier 70 meV lower. (c) ASE eon reader resume with the periodicity restored: adatom frozen on every axis. Each panel marks its own saddle (SP, star) and the saddles of the other panels. At this projection the saddles fall within a marker of each other; the routes differ in energy and in free modes.

6.2. The mask and the prefactor

The symmetric hop hides what a frozen axis does to a rate. A climbing-image band of seven images on the $2 \times 2 \times 2$ slab was converged to 0.01 eV/Å once with the intended masks and once with the adatom frozen, as the ASE eon reader returns it, interpolated with the constraints ignored. With the height held, the barrier is 0.366598 eV on a path of 2.90040 Å, and one imaginary mode of −4.08 meV has 0.987 of its weight on the lateral motion. The Vineyard prefactor of that constrained EMT Hessian is 3.24×10^{10} Hz [2], which is separate from a measured rate of Pt self-diffusion on this face [116]. Frozen, the barrier is 0.366584 eV on 2.95955 Å, the saddle geometries differ by 7.78×10^{-4} Å, and there is no imaginary mode, so the prefactor is undefined. The barrier alone would not reveal the error. Figure 3 places both bands on the (s, d) plane.

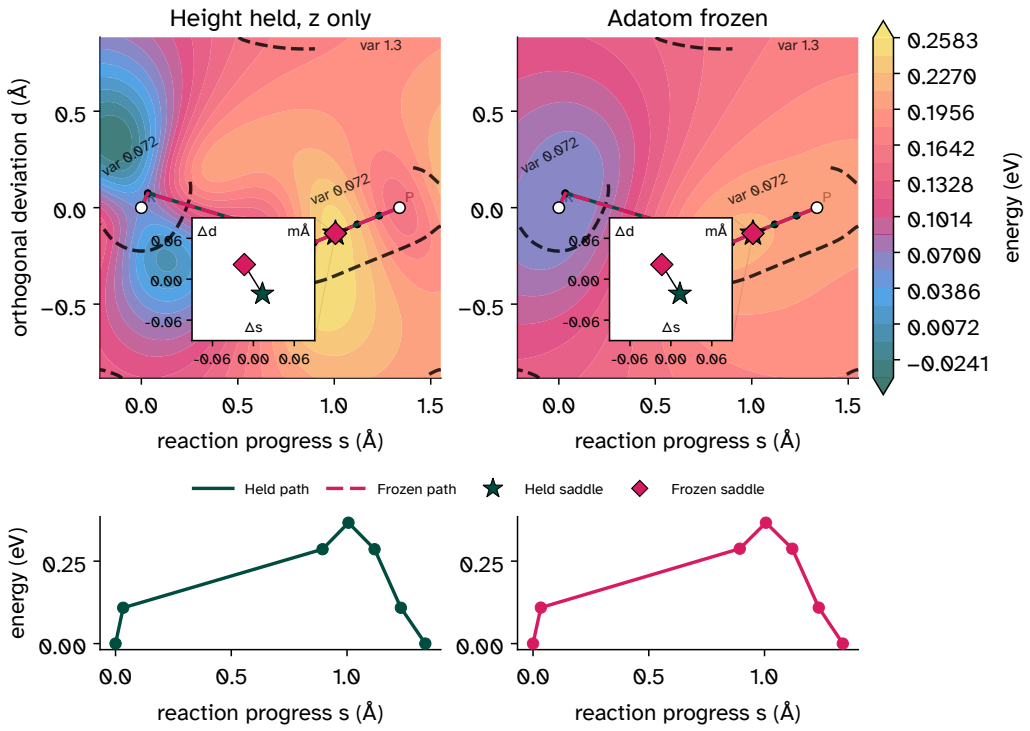


Figure 3: Climbing-image bands for a Pt adatom hop on Pt(100) with EMT, each mask on its own (s, d) map [115]. Height held: 0.366598 eV on 2.90040 Å. Frozen: 0.366584 eV on 2.95955 Å. The saddle geometries differ by 7.78×10^{-4} Å, and the frozen middle image has no imaginary mode.

6.3. eOn resuming its own band

The production path is eOn writing and reading CON through readcon-core. An eOn 3.6.0 client ran the Morse Pt potential on a 28-atom slab with the bottom nine atoms at mask 7, and a band of five movable images, again under steepest descent [32, 117]. Both the uninterrupted job and each resume take five steps and stop at that cap, so the band has not converged; the force at the last step is 0.83265 eV/Å. Both resumes reproduce the printed table of the uninterrupted job, maximum force 3.0658, 1.2007 and 0.59762 eV/Å and highest image 0.8305, 0.6911 and 0.6417 eV. After five steps the round-trip barrier is 0.63493 eV, identical to the uninterrupted band, with a largest minimum-image displacement of 8.9×10^{-15} Å. Six decimals shift the barrier by $+1.0 \times 10^{-6}$ eV and the images by up to 4.9×10^{-7} Å. Figure 4 shows the three bands, and Section S1 the per-image displacement at each

step. An L-BFGS resume, eOn's default band optimiser, ends at 0.6707 eV against 0.6336 eV for both printers, because the history is not in either file. A dimer from the round-trip geometry reproduces the uninterrupted saddle energy, −112.618941 eV, and curvature, −10.613681, and six decimals shift that energy by -3.0×10^{-6} eV [33].

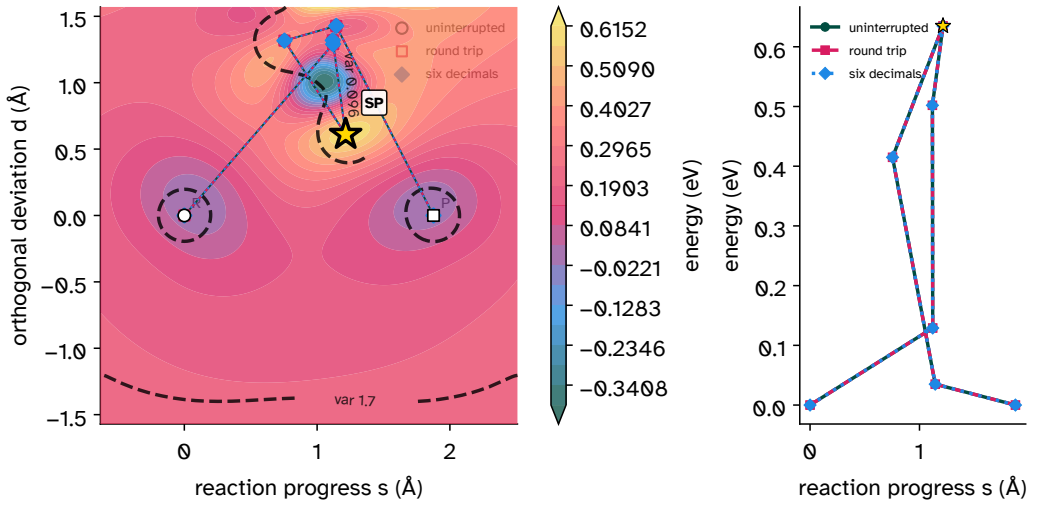


Figure 4: Final bands of the interrupted Morse band on the (s, d) plane [115, 32, 117]. The round-trip barrier is 0.63493 eV, identical to the uninterrupted band. The job stops at five steepest-descent steps.

6.4. An asymmetric band resumed

The Pt hops above are symmetric, so a band can pass through the saddle on the straight line. A palladium adatom on a $3 \times 3 \times 3$ Al(111) slab with EMT moves from an fcc hollow to an hcp hollow, and the endpoints differ by −11.3 meV. Mask 7 fixes the lowest aluminium layer, and the fast inertial relaxation engine (FIRE) drives a band of five movable images with a spring of 0.1 eV/Å^2 to 0.02 eV/Å [61]. With the palladium free the band converges in 31 steps to a forward barrier of 19.85 meV and a reverse barrier of 31.18 meV. With its height held by mask 4 it converges in 26 steps to 15.02 and 21.51 meV, so a lost or added height constraint moves the forward barrier by 4.83 meV, a quarter of its value. The converged free band, written through the canonical writer and read back, returns every coordinate bit for bit. Resumed from it at the same threshold, FIRE takes no step, the positions stay bit-identical,

and the energy differs from the pre-restart value by 8.9×10^{-15} eV, at the rounding level of the force field.

6.5. The velocity unit and identity

On the 28-atom slab the kinetic energy is 0.888 eV in the natural unit and 92.02 eV if the same numbers are read as Å/fs, a factor of 103.64. One velocity Verlet step of 1 fs then misplaces atoms by an RMSD of 0.096 Å (Section S4). For identity, the public HCN and HNC endpoints were written with ids 2, 0 and 1 and an x-only mask on hydrogen [118, 119]. readcon-core returns that mask, those ids and the declared unit. The ASE eon reader does not accept a velocity block and, on the coordinate block, reports hydrogen frozen on every axis with no id (Section S2).

7. Performance

7.1. Cost of a read

A resume reads its checkpoint once per force call at most, so a read has to stay cheap beside that call. The timings use one file on one Intel Xeon processor of family 6, model 207, with 8 hardware threads, rustc 1.88, g++ 13.3 and Python 3.12. The layer and thread timings use a 167.71 MB file of 10,000 copies of one 218-atom frame, read from a warm page cache, so the rate is the cost of this frame. The peer comparison and the readcon-db ingest use a 10,000-frame velocity Verlet EMT trajectory of the same frame at 1 fs and 300 K, in which the two free hydrogen atoms move and every frame is distinct; its CON file is 92.03 MB. An interval is a bias-corrected and accelerated 95 percent interval of the median [120, 121, 122, 123]. Table 12 and Figure 5 give the layers.

Table 12: Median timings on the 10,000-frame file. Frames per second are 10,000 divided by the median. The automatic Rust interval is $[0.115, 0.137]$ s and the Python interval is $[1.216, 1.304]$ s.

Layer	Threads	Time (s)	MB/s	frames/s
Rust core	automatic	0.126	1,336	79,681
C++ wrapper	automatic	0.133	1,260	75,103
Rust core	1	0.484	347	20,676
C++ wrapper	1	0.485	346	20,624
Python objects	1	1.235	136	8,100

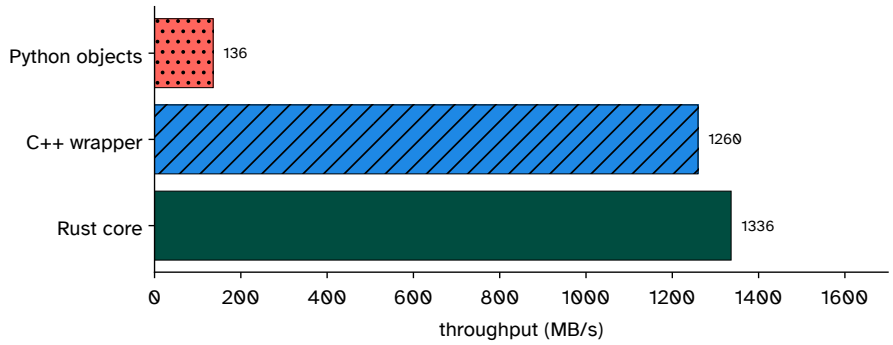


Figure 5: Median throughput on the 167.71 MB file, warm cache. The Rust core and the C++ wrapper use the automatic policy. The Python bar includes one object per atom.

The C++ wrapper eOn calls costs nothing measurable over the core: the automatic Rust time is 0.904 of the C++ time, interval 0.815 to 0.993, and on one thread the ratio is 0.991. One frame at the automatic rate costs 12.6 μs, against 0.304 ms for one Morse call and 52.6 μs for one EMT call on the 28-atom slab, so the whole file costs about 413 Morse calls; the i-PI 3.0 measurements use the same force-call framing [86]. Table 13 times each stage of the read on its own, 30 repeats per thread count with 90 at one thread. The full read falls from 0.445 s on one thread to 0.126 s on eight, a speedup of 3.53 with interval 3.48 to 3.56, and the in-memory parse alone reaches 4.83, interval 4.71 to 4.89. The token walk and the numeric parse scale to 7.55 and 7.60, the boundary scan stays at 0.033 s on every thread count, and assembly peaks at 2.77 on four threads. The Amdahl bound we drew earlier counted the string load and the boundary scan as the serial fraction, $f = (0.0824 + 0.0332)/0.445 = 0.260$, which gives 2.84 on 8 threads. That bound is conservative: the timed read maps the file and validates it in place, so the string load is not on its path. The boundary scan alone gives $f = 0.075$ and a bound of 5.25, and the measured speedup implies an effective serial fraction of 0.181. Every thread count returns the same frames. Python flattens at 1.00, 1.22, 1.27 and 1.15, because object construction takes the interpreter lock.

7.2. Peers on the same frames

Table 14 sets the one-worker Python read and write beside five peers on the 10,000-frame trajectory, as 30 interleaved pairs per cell with a Holm adjustment, from a cold and a warm page cache [124, 63, 125, 126, 127].

Table 13: Warm-cache stage timings on the 10,000-frame repeated file: median at one thread and speedup at 8 threads with its interval. The full read is the library call; the other rows are isolated passes.

Stage	1 thread (s)	Speedup at 8	Interval
full read	0.4449	3.53	[3.48, 3.56]
in-memory parse	0.4255	4.83	[4.71, 4.89]
numeric parse	0.3477	7.60	[7.01, 7.70]
token walk	0.1679	7.55	[7.09, 7.67]
string load	0.0824	0.93	[0.92, 0.94]
assembly	0.0394	2.43	[2.39, 2.46]
boundary scan	0.0332	0.99	[0.99, 1.00]
library file load	0.0262	1.01	[1.01, 1.02]
UTF-8 validation	0.0203	0.76	[0.75, 0.79]

Each ratio is the readcon-core time over the peer time, so a value below 1 means readcon-core finished sooner, and every adjusted p-value is 0.002. The peers return different amounts of the frame. readcon-core parses the CON text and returns one object per atom with the mask, the id and the declared units, in 1.22 s cold and 1.15 to 1.20 s warm. h5py returns the binary64 position array without parsing anything, and the ratios of 37.2 cold and 87.2 warm are the distance between a text parse and a binary load. chemfiles parses XYZ text with symbols and coordinates only, at 2.49 cold and 2.44 warm. The ASE trajectory is binary and returns ASE atoms with the mask and without ids, at 0.960 cold and 0.940 warm. MDAnalysis returns single-precision coordinates with the declared unit from H5MD, at 0.208 and 0.200, and ASE returns ASE atoms with the mask and ids by column convention from extended XYZ, at 0.282 and 0.281. Writing from objects already in memory, readcon-core takes 1.55 to 1.62 s, 0.19 to 0.24 of the ASE extended XYZ and MDAnalysis writers, about twice chemfiles and the ASE trajectory, and about 60 times h5py writing one array. On the repeated file the ratios are within 0.16 of these except the ASE trajectory read, at 1.40 cold and 1.31 warm, and h5py. Table S2 in the supplement lists the earlier payloads.

7.3. Parallel read, write and ingest

The output of every parallel path is independent of the rank count: each file read or written has the sha256 of the serial one, and each ingested store has the canonical dump of the serial ingest, on every rank count measured.

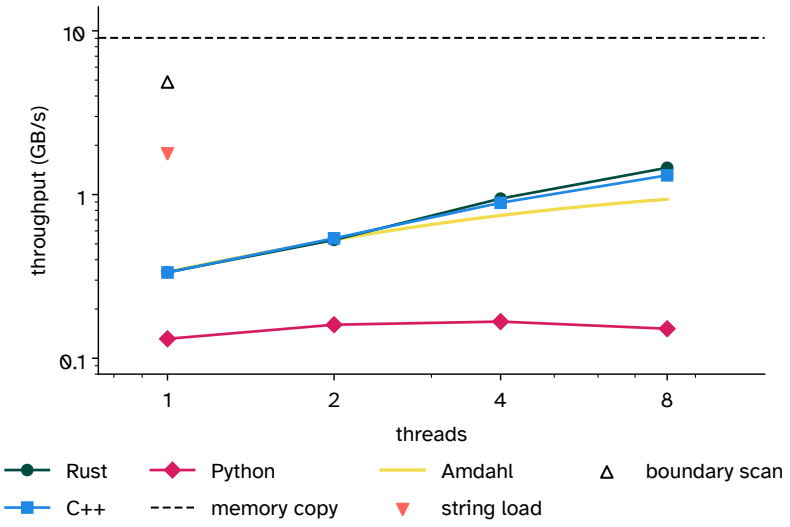


Figure 6: Parse throughput at 1, 2, 4 and 8 threads on the repeated 218-atom frame. The dashed line is the hot-cache memory copy at 9.06 GB/s. Points are the layer timings of Table 12. The Amdahl curve counts the string load and the boundary scan as serial, and is a conservative bound because the string load is not on the timed path. Triangles mark the string-load and boundary-scan rates.

The MPI paths take the caller's communicator and give each rank a contiguous block of frames, so the rank-ordered concatenation is the serial read. On a 1.07 GB file of 3,167,382 frames, with MPICH 4.2.3 and one thread per rank, a read with every rank mapping a shared file falls from 9.39 s on one rank to 1.63 s on eight, a speedup of 5.74, and rank 0 scattering byte ranges reaches 4.71 (Figure 7a). The collective write places each rank's bytes at an exclusive-scan offset; through MPI-IO it falls from 7.94 s to 1.22 s, a speedup of 6.52, and gathered to rank 0 it reaches 4.25. readcon-db [113] ingests four copies of a 1.07 GB trajectory, batch 4096, in Table 15 and Figure 7b. With one writer the ingest falls from 30.6 s on one rank to 15.3 s on eight, a speedup of 2.00, and with four shards from 32.5 s to 17.1 s, a speedup of 1.90. At 16 ranks the two modes take 35.2 s and 43.5 s; the serial ingest on the same host takes 22.8 to 25.9 s up to eight ranks and 51.6 s at 16.

On one thread, readcon-db ingests the 10,000-frame trajectory in 0.579 s, 17,260 frames per second, against 3,373 for the ASE database in one transaction and 592 when it commits every row; the paired ratios are 0.193, interval 0.190 to 0.196, and 0.0337, interval 0.0324 to 0.0346. An ingest of 80,000

Table 14: Paired ratios on the 10,000-frame trajectory, readcon-core time over peer time, one worker. Each cell is the median of 30 interleaved pairs with its bias-corrected and accelerated 95 percent interval. Cold repeats drop the page cache before each call.

Peer	Returns		Cold	Warm
ASE extended XYZ	ASE atoms	read	0.282 [0.280, 0.283]	0.281 [0.281, 0.281]
		write	0.196 [0.194, 0.197]	0.188 [0.187, 0.190]
ASE trajectory	ASE atoms	read	0.960 [0.952, 0.964]	0.940 [0.936, 0.948]
		write	2.06 [2.05, 2.08]	2.02 [2.00, 2.04]
chemfiles XYZ	parsed XYZ	read	2.49 [2.47, 2.51]	2.44 [2.44, 2.45]
		write	1.99 [1.98, 2.00]	1.98 [1.97, 1.99]
h5py H5MD	raw array	read	37.2 [35.5, 37.7]	87.2 [86.1, 88.1]
		write	64.4 [63.6, 65.2]	59.1 [58.7, 59.3]
MDAnalysis H5MD	float32 array	read	0.208 [0.204, 0.210]	0.200 [0.198, 0.201]
		write	0.235 [0.233, 0.236]	0.225 [0.223, 0.227]

Table 15: readcon-db MPI ingest of four copies of a 1.07 GB trajectory (30,965 frames each), batch 4096, MPICH 4.2.3, means of five runs in seconds. Serial is the serial ingest that rank 0 runs on the same host right after each MPI run. Every MPI corpus has the canonical dump of the serial one.

	One writer			Four shards		
Ranks	MPI	Speedup	Serial	MPI	Speedup	Serial
1	30.62	1.00	22.82	32.54	1.00	23.19
2	22.59	1.36	23.28	24.67	1.32	24.89
4	17.68	1.73	24.12	22.58	1.44	24.46
8	15.28	2.00	24.21	17.08	1.90	25.93
16	35.17	0.87	51.61	43.52	0.75	51.11

distinct frames, 1.34 GB, grows the store past its 2 GiB default map without intervention, at 12,344 frames per second.

Weak scaling, at a fixed number of frames per rank, is [PENDING: weak scaling, cloud run 04].

8. Limitations

CON 3.1 is a text format for the frames a search checkpoints, not for analysis trajectories. Text costs space and a parse (Table 7), which the core pays at 1.3 GB/s. Indexed random access and campaign storage are readcon-db's part of the design [113]. A frame specifies geometry and its labels,

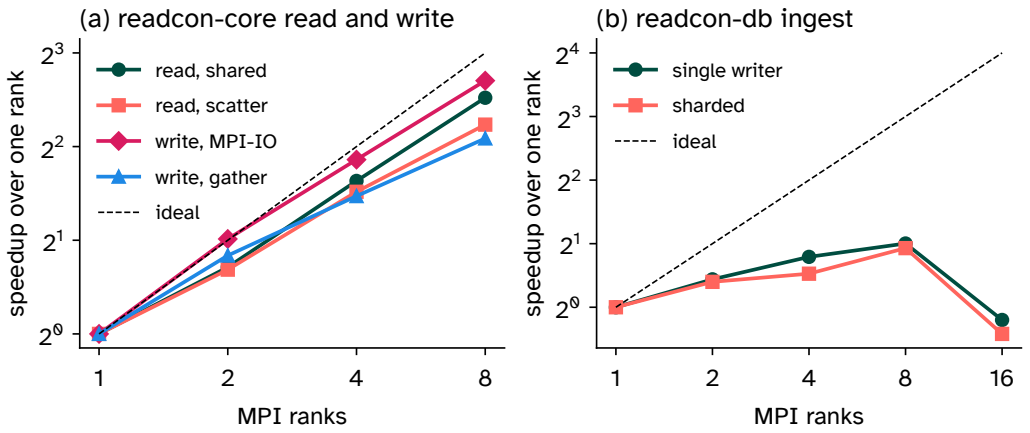


Figure 7: Strong scaling with MPICH 4.2.3, one thread per rank, medians of five runs for readcon-core and means of five for readcon-db. (a) Read and write of a 1.07 GB file of 3,167,382 frames. (b) Ingest of four copies of a 1.07 GB trajectory into readcon-db [113] with one writer or four shards. The dashed line is ideal speedup. Every output has the bytes of the serial one.

not optimiser state, so an L-BFGS resume restarts its history; a key for that history is the natural next minor version. The conformance corpus fixes what a conforming reader decides on each listed file, and it says nothing about a reader on files outside it. A single core behind every binding makes the bindings agree by construction, which is a property of this implementation and not evidence about a second one. The resume studies are one symmetric and one asymmetric EMT hop and one Morse band; they show what a lost field does to a saddle, and they are not a survey of reactions. The peer comparisons hold for the library versions named in Section 2.

9. Conclusions

A geometry file in a rare-event campaign is the restart state of the search, and it needs a per-axis mask, a persistent id, a declared time unit, binary64 coordinates with their periodicity, and a version two readers agree on. On one masked slab frame, no common format keeps all five, and the losses change the computed saddle: a POSCAR resume ends 70 meV lower, and an ASE eon resume loses two modes and the periodicity. Five readers of one extended XYZ file give four answers. CON 3.1 specifies the text eOn already writes, its corpus decides what a reader must accept, and readcon-core gives

every language the same parser. A band resumed through it matches the uninterrupted band to the floor set by restarting the force field, and a frame costs 12.6 μs to read.

Data availability

The version 0.17.0 archive is https://doi.org/10.5281/zenodo.23302640 [105], and the concept record is https://doi.org/10.5281/zenodo.18753567 [128]. A citation to the software is owed to that archived record [129, 130, 131, 132]. Publishing a finished saddle remains the job of a repository interface, a materials archive or a reaction database, downstream of this checkpoint [133, 134, 135, 136, 137]. The HCN coordinates used in the cross-reader check are the public endpoints of that isomerisation [118, 119]. The specification, the grammar, the corpus, the failure-case scripts and their outputs are the supplementary material. The accompanying notes record the restart displacements, the format round trips, the frame bytes, the velocity step, the illegal-file classes, the database hashes, and the stored ingest timings.

CRediT authorship contribution statement

Rohit Goswami: Conceptualization, Methodology, Software, Visualization, Writing - Original draft preparation, Writing - Reviewing and Editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work received no funding.

Acknowledgements

The author thanks Hannes Jónsson for long discussions on saddle searches and on what an eOn checkpoint has to carry, and Miha Gunde for discussions on matching atoms along a reaction path. Ruhila Goswami designed the colour palette of the figures. The library was written with the patience of the author's family, in the company of the turtles, the garden cats, the birds and the rest of the household.

References

[1] H. Eyring, The activated complex in chemical reactions, The Journal of Chemical Physics 3 (2) (1935) 107–115. doi:10.1063/1.1749604.
URL https://pubs.aip.org/jcp/article/3/2/107/203352/The-Activated-Complex-in-Chemical-Reactions

[2] G. H. Vineyard, Frequency factors and isotope effects in solid state rate processes, Journal of Physics and Chemistry of Solids 3 (1-2) (1957) 121–127. doi:10.1016/0022-3697(57)90059-8.
URL https://linkinghub.elsevier.com/retrieve/pii/0022369757900598

[3] P. Hänggi, P. Talkner, M. Borkovec, Reaction-rate theory: fifty years after Kramers, Reviews of Modern Physics 62 (2) (1990) 251–341. doi:10.1103/revmodphys.62.251.
URL https://link.aps.org/doi/10.1103/RevModPhys.62.251

[4] G. Henkelman, G. Jóhannesson, H. Jónsson, Methods for finding saddle points and minimum energy paths, in: Theoretical Methods in Condensed Phase Chemistry, Kluwer Academic Publishers, 2005, Ch. 10, pp. 269–302. doi:10.1007/0-306-46949-9_10.
URL http://link.springer.com/10.1007/0-306-46949-9_10

[5] H. Jónsson, G. Mills, K. W. Jacobsen, Nudged elastic band method for finding minimum energy paths of transitions, in: Classical and Quantum Dynamics in Condensed Phase Simulations, WORLD SCIENTIFIC, 1998, pp. 385–404. doi:10.1142/9789812839664_0016.
URL http://www.worldscientific.com/doi/abs/10.1142/9789812839664_0016

[6] G. Henkelman, B. Uberuaga, H. Jónsson, A climbing image nudged elastic band method for finding saddle points and minimum energy paths, The Journal of Chemical Physics 113 (22) (2000) 9901–9904. doi:10.1063/1.1329672.
URL https://pubs.aip.org/jcp/article/113/22/9901/185050/A-climbing-image-nudged-elastic-band-method-for

[7] V. Ásgeirsson, B. O. Birgisson, R. Bjornsson, U. Becker, F. Neese, C. Riplinger, H. Jónsson, Nudged elastic band method for molecular reactions using energy-weighted springs combined with eigenvector following, Journal of Chemical Theory and Computation 17 (8) (2021) 4929–4945. doi:10.1021/acs.jctc.1c00462.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.1c00462

[8] S. Smidstrup, A. Pedersen, K. Stokbro, H. Jónsson, Improved initial guess for minimum energy path calculations, The Journal of Chemical Physics 140 (21) (2014) 214106. doi:10.1063/1.4878664.
URL https://pubs.aip.org/jcp/article/140/21/214106/566679/Improved-initial-guess-for-minimum-energy-path

[9] W. E, W. Ren, E. Vanden-Eijnden, String method for the study of rare events, Physical Review B 66 (5) (2002) 052301. doi:10.1103/physrevb.66.052301.
URL https://link.aps.org/doi/10.1103/PhysRevB.66.052301

[10] W. E, W. Ren, E. Vanden-Eijnden, Simplified and improved string method for computing the minimum energy paths in barrier-crossing events, The Journal of Chemical Physics 126 (16) (2007) 164103. doi:10.1063/1.2720838.
URL https://pubs.aip.org/jcp/article/126/16/164103/905415/Simplified-and-improved-string-method-for

[11] B. Peters, A. Heyden, A. T. Bell, A. Chakraborty, A growing string method for determining transition states: Comparison to the nudged elastic band and string methods, The Journal of Chemical Physics 120 (17) (2004) 7877–7886. doi:10.1063/1.1691018.
URL https://pubs.aip.org/jcp/article/120/17/7877/534771/A-growing-string-method-for-determining-transition

[12] G. Henkelman, H. Jónsson, A dimer method for finding saddle points on high dimensional potential surfaces using only first derivatives, The Journal of Chemical Physics 111 (15) (1999) 7010–7022. doi:10.1063/1.480097.
URL https://pubs.aip.org/jcp/article/111/15/7010/475160/A-dimer-method-for-finding-saddle-points-on-high

[13] A. Heyden, A. T. Bell, F. J. Keil, Efficient methods for finding transition states in chemical reactions: Comparison of improved dimer method and partitioned rational function optimization method, The Journal of Chemical Physics 123 (22) (2005) 224101. doi:10.1063/1.2104507.
URL https://pubs.aip.org/jcp/article/123/22/224101/776452/Efficient-methods-for-finding-transition-states-in

[14] J. Kästner, P. Sherwood, Superlinearly converging dimer method for transition state search, The Journal of Chemical Physics 128 (1) (2008) 014106. doi:10.1063/1.2815812.
URL https://pubs.aip.org/jcp/article/128/1/014106/928095/Superlinearly-converging-dimer-method-for

[15] R. A. Olsen, G. J. Kroes, G. Henkelman, A. Arnaldsson, H. Jónsson, Comparison of methods for finding saddle points without knowledge of the final states, The Journal of Chemical Physics 121 (20) (2004) 9776–9792. doi:10.1063/1.1809574.
URL https://pubs.aip.org/jcp/article/121/20/9776/532635/Comparison-of-methods-for-finding-saddle-points

[16] D. J. Wales, Exploring energy landscapes, Annual Review of Physical Chemistry 69 (1) (2018) 401–425. doi:10.1146/annurev-physchem-050317-021219.
URL https://www.annualreviews.org/doi/10.1146/annurev-physchem-050317-021219

[17] E. D. Hermes, K. Sargsyan, H. N. Najm, J. Zádor, Accelerated saddle point refinement through full exploitation of partial Hessian diagonalization, Journal of Chemical Theory and Computation 15 (11) (2019) 6536–6549. doi:10.1021/acs.jctc.9b00869.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.9b00869

[18] E. D. Hermes, K. Sargsyan, H. N. Najm, J. Zádor, Geometry optimization speedup through a geodesic approach to internal coordinates, The Journal of Chemical Physics 155 (9) (2021) 094105. doi:10.1063/5.0060146.
URL https://pubs.aip.org/jcp/article/155/9/094105/199820/Geometry-optimization-speedup-through-a-geodesic

[19] E. D. Hermes, K. Sargsyan, H. N. Najm, J. Zádor, Sella, an open-source automation-friendly molecular saddle point optimizer, Journal of Chemical Theory and Computation 18 (11) (2022) 6974–6988. doi:10.1021/acs.jctc.2c00395.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.2c00395

[20] A. F. Voter, Hyperdynamics: Accelerated molecular dynamics of infrequent events, Physical Review Letters 78 (20) (1997) 3908–3911. doi:10.1103/physrevlett.78.3908.
URL https://link.aps.org/doi/10.1103/PhysRevLett.78.3908

[21] A. F. Voter, F. Montalenti, T. C. Germann, Extending the time scale in atomistic simulation of materials, Annual Review of Materials Research 32 (1) (2002) 321–346. doi:10.1146/annurev.matsci.32.112601.141541.
URL https://www.annualreviews.org/doi/10.1146/annurev.matsci.32.112601.141541

[22] R. A. Miron, K. A. Fichthorn, Accelerated molecular dynamics with the bond-boost method, The Journal of Chemical Physics 119 (12) (2003) 6210–6216. doi:10.1063/1.1603722.
URL https://pubs.aip.org/jcp/article/119/12/6210/451910/Accelerated-molecular-dynamics-with-the-bond-boost

[23] G. Henkelman, H. Jónsson, Long time scale kinetic Monte Carlo simulations without lattice approximation and predefined event table, The Journal of Chemical Physics 115 (21) (2001) 9657–9666. doi:10.1063/1.1415500.
URL https://pubs.aip.org/jcp/article/115/21/9657/453229/Long-time-scale-kinetic-Monte-Carlo-simulations

[24] A. F. Voter, Introduction to the kinetic Monte Carlo method, in: Radiation Effects in Solids, Springer Netherlands, 2007, Ch. 1, pp. 1–23.

doi:10.1007/978-1-4020-5295-8_1.
URL http://link.springer.com/10.1007/978-1-4020-5295-8_1

[25] L. Xu, G. Henkelman, Adaptive kinetic Monte Carlo for first-principles accelerated dynamics, The Journal of Chemical Physics 129 (11) (2008) 114104. doi:10.1063/1.2976010.
URL https://pubs.aip.org/jcp/article/129/11/114104/296009/Adaptive-kinetic-Monte-Carlo-for-first-principles

[26] M. A. Novotny, Monte Carlo algorithms with absorbing Markov chains: Fast local algorithms for slow dynamics, Physical Review Letters 74 (1) (1995) 1–5. doi:10.1103/physrevlett.74.1.
URL https://link.aps.org/doi/10.1103/PhysRevLett.74.1

[27] A. F. Voter, Parallel replica method for dynamics of infrequent events, Physical Review B 57 (22) (1998) R13985–R13988. doi:10.1103/physrevb.57.r13985.
URL https://link.aps.org/doi/10.1103/PhysRevB.57.R13985

[28] D. Perez, E. D. Cubuk, A. Waterland, E. Kaxiras, A. F. Voter, Long-time dynamics through parallel trajectory splicing, Journal of Chemical Theory and Computation 12 (1) (2015) 18–28. doi:10.1021/acs.jctc.5b00916.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.5b00916

[29] D. J. Wales, J. P. K. Doye, Global optimization by basin-hopping and the lowest energy structures of Lennard-Jones clusters containing up to 110 atoms, The Journal of Physical Chemistry A 101 (28) (1997) 5111–5116. doi:10.1021/jp970984n.
URL https://pubs.acs.org/doi/10.1021/jp970984n

[30] S. Goedecker, Minima hopping: An efficient search method for the global minimum of the potential energy surface of complex molecular systems, The Journal of Chemical Physics 120 (21) (2004) 9911–9917. doi:10.1063/1.1724816.
URL https://pubs.aip.org/jcp/article/120/21/9911/533554/Minima-hopping-An-efficient-search-method-for-the

[31] S. T. Chill, M. Welborn, R. Terrell, L. Zhang, J.-C. Berthet, A. Pedersen, H. Jónsson, G. Henkelman, EON: software for long time simulations of atomic scale systems, Modelling and Simulation in Materials

Science and Engineering 22 (5) (2014) 055002. doi:10.1088/0965-0393/22/5/055002.
URL https://iopscience.iop.org/article/10.1088/0965-0393/22/5/055002

[32] R. Goswami, S. Chill, R. Terrell, G. Henkelman, M. Welborn, L. Zhang, A. Pedersen, t brink, E. Edelmann, J. Claude, Alejandro, S. H. Jung, S. A. Ghasemi, chemist29, satishskamath, Maxim, T. Dijkhuis, via9a, TheochemUI/eOn: v3.6.0, Zenodo, software, version 3.6.0 (2026). doi:10.5281/zenodo.23166734.

[33] R. Goswami, Efficient exploration of chemical kinetics, arXiv:2510.21368 (2025). doi:10.48550/arxiv.2510.21368.

[34] D. P. Anderson, BOINC: A system for public-resource computing and storage, in: Fifth IEEE/ACM International Workshop on Grid Computing, IEEE, 2004, pp. 4–10. doi:10.1109/grid.2004.14.
URL http://ieeexplore.ieee.org/document/1382809/

[35] A. Pedersen, H. Jónsson, Distributed implementation of the adaptive kinetic Monte Carlo method, Mathematics and Computers in Simulation 80 (7) (2010) 1487–1498. doi:10.1016/j.matcom.2009.02.010.
URL https://linkinghub.elsevier.com/retrieve/pii/S0378475409000500

[36] R. Goswami, M. Masterov, S. Kamath, A. Peña-Torres, H. Jónsson, Efficient implementation of Gaussian process regression accelerated saddle point searches with application to molecular reactions, Journal of Chemical Theory and Computation 21 (16) (2025) 7935–7943. doi:10.1021/acs.jctc.5c00866.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.5c00866

[37] O.-P. Koistinen, V. Ásgeirsson, A. Vehtari, H. Jónsson, Nudged elastic band calculations accelerated with Gaussian process regression based on inverse interatomic distances, Journal of Chemical Theory and Computation 15 (12) (2019) 6738–6751. doi:10.1021/acs.jctc.9b00692.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.9b00692

[38] O.-P. Koistinen, V. Ásgeirsson, A. Vehtari, H. Jónsson, Minimum mode saddle point searches using Gaussian process regression with inverse-

distance covariance function, Journal of Chemical Theory and Computation 16 (1) (2019) 499–509. doi:10.1021/acs.jctc.9b01038.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.9b01038

[39] A. A. Peterson, Acceleration of saddle-point searches with machine learning, The Journal of Chemical Physics 145 (7) (2016) 074106. doi:10.1063/1.4960708.
URL https://pubs.aip.org/jcp/article/145/7/074106/810392/Acceleration-of-saddle-point-searches-with-machine

[40] R. D. Peng, Reproducible research in computational science, Science 334 (6060) (2011) 1226–1227. doi:10.1126/science.1213847.
URL https://www.science.org/doi/10.1126/science.1213847

[41] D. C. Ince, L. Hatton, J. Graham-Cumming, The case for open computer programs, Nature 482 (7386) (2012) 485–488. doi:10.1038/nature10836.
URL https://www.nature.com/articles/nature10836

[42] K. Hinsen, Caring for your data, Computing in Science & Engineering 14 (6) (2012) 70–74. doi:10.1109/mcse.2012.108.
URL http://ieeexplore.ieee.org/document/6341744/

[43] K. Hinsen, Dealing with software collapse, Computing in Science & Engineering 21 (3) (2019) 104–108. doi:10.1109/mcse.2019.2900945.
URL https://ieeexplore.ieee.org/document/8701540/

[44] H. Jónsson, Simulation of surface processes, Proceedings of the National Academy of Sciences 108 (3) (2011) 944–949. doi:10.1073/pnas.1006670108.
URL https://pnas.org/doi/full/10.1073/pnas.1006670108

[45] M. Gunde, N. Salles, A. Hémeryck, L. Martin-Samos, IRA: A shape matching approach for recognition and comparison of generic atomic patterns, Journal of Chemical Information and Modeling 61 (11) (2021) 5446–5457. doi:10.1021/acs.jcim.1c00567.
URL https://pubs.acs.org/doi/10.1021/acs.jcim.1c00567

[46] D. Perez, B. P. Uberuaga, A. F. Voter, The parallel replica dynamics method – Coming of age, Computational Materials Science 100 (2015)

90–103. doi:10.1016/j.commatsci.2014.12.011.
URL https://linkinghub.elsevier.com/retrieve/pii/S0927025614008489

[47] R. Terrell, M. Welborn, S. T. Chill, G. Henkelman, Database of atomistic reaction mechanisms with application to kinetic Monte Carlo, The Journal of Chemical Physics 137 (1) (2012) 014105. doi:10.1063/1.4730746.
URL https://pubs.aip.org/jcp/article/137/1/014105/191477/Database-of-atomistic-reaction-mechanisms-with

[48] E. Tiesinga, P. J. Mohr, D. B. Newell, B. N. Taylor, CODATA recommended values of the fundamental physical constants: 2018, Reviews of Modern Physics 93 (2) (2021) 025010. doi:10.1103/revmodphys.93.025010.
URL https://link.aps.org/doi/10.1103/RevModPhys.93.025010

[49] S. McKeever, O. Bennich-Björkman, O.-A. Salah, Unit of measurement libraries, their popularity and suitability, Software: Practice and Experience 51 (4) (2020) 711–734. doi:10.1002/spe.2926.
URL https://onlinelibrary.wiley.com/doi/10.1002/spe.2926

[50] W. C. Swope, H. C. Andersen, P. H. Berens, K. R. Wilson, A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters, The Journal of Chemical Physics 76 (1) (1982) 637–649. doi:10.1063/1.442716.
URL https://pubs.aip.org/jcp/article/76/1/637/397800/A-computer-simulation-method-for-the-calculation

[51] G. Bussi, D. Donadio, M. Parrinello, Canonical sampling through velocity rescaling, The Journal of Chemical Physics 126 (1) (2007) 014101. doi:10.1063/1.2408420.
URL https://pubs.aip.org/jcp/article/126/1/014101/186581/Canonical-sampling-through-velocity-rescaling

[52] R. Gowers, M. Linke, J. Barnoud, T. Reddy, M. Melo, S. Seyler, J. Domański, D. Dotson, S. Buchoux, I. Kenney, O. Beckstein, MDAnalysis: A Python package for the rapid analysis of molecular dynamics simulations, in: Proceedings of the Python in Science Conference, SciPy,

2016, pp. 98–105. doi:10.25080/majora-629e541a-00e.
URL https://doi.curvenote.com/10.25080/Majora-629e541a-00e

[53] P. Murray-Rust, H. S. Rzepa, Chemical markup, XML, and the worldwide web. 1. basic principles, Journal of Chemical Information and Computer Sciences 39 (6) (1999) 928–942. doi:10.1021/ci990052b.
URL https://pubs.acs.org/doi/10.1021/ci990052b

[54] K. Hinsen, MOSAIC: A data model and file formats for molecular simulations, Journal of Chemical Information and Modeling 54 (1) (2013) 131–137. doi:10.1021/ci400599y.
URL https://pubs.acs.org/doi/10.1021/ci400599y

[55] P. d. Buyl, P. H. Colberg, F. Höfling, H5MD: A structured, efficient, and portable file format for molecular data, Computer Physics Communications 185 (6) (2014) 1546–1553. doi:10.1016/j.cpc.2014.01.018.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465514000447

[56] M. Folk, G. Heber, Q. Koziol, E. Pourmal, D. Robinson, An overview of the HDF5 technology suite and its applications, in: Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases, ACM, 2011, pp. 36–47. doi:10.1145/1966895.1966900.
URL https://dl.acm.org/doi/10.1145/1966895.1966900

[57] S. R. Bahn, K. W. Jacobsen, An object-oriented scripting interface to a legacy electronic structure code, Computing in Science & Engineering 4 (3) (2002) 56–66. doi:10.1109/5992.998641.
URL http://ieeexplore.ieee.org/document/998641/

[58] A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. B. Jensen, J. Kermode, J. R. Kitchin, E. L. Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. B. Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz, O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng, K. W. Jacobsen, The atomic simulation environment—a Python library for working with atoms, Jour-

nal of Physics: Condensed Matter 29 (27) (2017) 273002. doi:10.1088/1361-648x/aa680e.
URL https://iopscience.iop.org/article/10.1088/1361-648X/aa680e

[59] J. Nocedal, Updating quasi-Newton matrices with limited storage, Mathematics of Computation 35 (151) (1980) 773–782. doi:10.1090/s0025-5718-1980-0572855-7.
URL https://www.ams.org/mcom/1980-35-151/S0025-5718-1980-0572855-7/

[60] D. Sheppard, R. Terrell, G. Henkelman, Optimization methods for finding minimum energy paths, The Journal of Chemical Physics 128 (13) (2008) 134106. doi:10.1063/1.2841941.
URL https://pubs.aip.org/jcp/article/128/13/134106/977389/Optimization-methods-for-finding-minimum-energy

[61] E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, P. Gumbsch, Structural relaxation made simple, Physical Review Letters 97 (17) (2006) 170201. doi:10.1103/physrevlett.97.170201.
URL https://link.aps.org/doi/10.1103/PhysRevLett.97.170201

[62] G. Fraux, L. Kimms, J. Fine, G. P. Barletta, M. Dimura, F. Coudert, Pelsa, M. Levesque, S. Maiti, S. Guionniere, Jmintser, chemfiles/chemfiles: Version 0.10.4, Zenodo, software, version 0.10.4 (2023). doi:10.5281/zenodo.7904565.

[63] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, O. Beckstein, MDAnalysis: A toolkit for the analysis of molecular dynamics simulations, Journal of Computational Chemistry 32 (10) (2011) 2319–2327. doi:10.1002/jcc.21787.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.21787

[64] G. Kresse, J. Furthmüller, Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set, Physical Review B 54 (16) (1996) 11169–11186. doi:10.1103/physrevb.54.11169.
URL https://link.aps.org/doi/10.1103/PhysRevB.54.11169

[65] G. Kresse, J. Furthmüller, Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set, Computational Materials Science 6 (1) (1996) 15–50. doi:10.1016/0927-0256(96)00008-0.
URL https://linkinghub.elsevier.com/retrieve/pii/0927025696000080

[66] M. Lundborg, R. Apostolov, D. Spångberg, A. Gärdenäs, D. van der Spoel, E. Lindahl, An efficient and extensible format, library, and API for binary trajectory data from molecular simulations, Journal of Computational Chemistry 35 (3) (2013) 260–269. doi:10.1002/jcc.23495.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.23495

[67] D. Spångberg, D. S. D. Larsson, D. v. d. Spoel, Trajectory NG: portable, compressed, general molecular dynamics trajectories, Journal of Molecular Modeling 17 (10) (2011) 2669–2685. doi:10.1007/s00894-010-0948-5.
URL http://link.springer.com/10.1007/s00894-010-0948-5

[68] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, E. Lindahl, GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers, SoftwareX 1-2 (2015) 19–25. doi:10.1016/j.softx.2015.06.001.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352711015000059

[69] B. R. Brooks, C. L. Brooks, A. D. Mackerell, L. Nilsson, R. J. Petrella, B. Roux, Y. Won, G. Archontis, C. Bartels, S. Boresch, A. Caflisch, L. Caves, Q. Cui, A. R. Dinner, M. Feig, S. Fischer, J. Gao, M. Hodoscek, W. Im, K. Kuczera, T. Lazaridis, J. Ma, V. Ovchinnikov, E. Paci, R. W. Pastor, C. B. Post, J. Z. Pu, M. Schaefer, B. Tidor, R. M. Venable, H. L. Woodcock, X. Wu, W. Yang, D. M. York, M. Karplus, CHARMM: The biomolecular simulation program, Journal of Computational Chemistry 30 (10) (2009) 1545–1614. doi:10.1002/jcc.21287.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.21287

[70] R. T. McGibbon, K. A. Beauchamp, M. P. Harrigan, C. Klein, J. M. Swails, C. X. Hernández, C. R. Schwantes, L.-P. Wang, T. J. Lane, V. S.

Pande, MDTraj: A modern open library for the analysis of molecular dynamics trajectories, Biophysical Journal 109 (8) (2015) 1528–1532. doi:10.1016/j.bpj.2015.08.015.
URL https://linkinghub.elsevier.com/retrieve/pii/S0006349515008267

[71] R. Rew, G. Davis, NetCDF: an interface for scientific data access, IEEE Computer Graphics and Applications 10 (4) (1990) 76–82. doi:10.1109/38.56302.
URL http://ieeexplore.ieee.org/document/56302/

[72] D. A. Case, T. E. Cheatham, T. Darden, H. Gohlke, R. Luo, K. M. Merz, A. Onufriev, C. Simmerling, B. Wang, R. J. Woods, The Amber biomolecular simulation programs, Journal of Computational Chemistry 26 (16) (2005) 1668–1688. doi:10.1002/jcc.20290.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.20290

[73] S. A. Brown, M. Folk, G. Goucher, R. Rew, P. F. Dubois, Software for portable scientific data management, Computers in Physics 7 (3) (1993) 304–308. doi:10.1063/1.4823180.
URL https://pubs.aip.org/cip/article/7/3/304/281673/Software-for-Portable-Scientific-Data-Management

[74] W. F. Godoy, N. Podhorszki, R. Wang, C. Atkins, G. Eisenhauer, J. Gu, P. Davis, J. Choi, K. Germaschewski, K. Huck, A. Huebl, M. Kim, J. Kress, T. Kurc, Q. Liu, J. Logan, K. Mehta, G. Ostrouchov, M. Parashar, F. Poeschel, D. Pugmire, E. Suchyta, K. Takahashi, N. Thompson, S. Tsutsumi, L. Wan, M. Wolf, K. Wu, S. Klasky, ADIOS 2: The Adaptable Input Output System. A framework for high-performance data management, SoftwareX 12 (2020) 100561. doi:10.1016/j.softx.2020.100561.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352711019302560

[75] H. M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T. N. Bhat, H. Weissig, I. N. Shindyalov, P. E. Bourne, The protein data bank, Nucleic Acids Research 28 (1) (2000) 235–242. doi:10.1093/nar/28.1.235.
URL https://academic.oup.com/nar/article-lookup/doi/10.1093/nar/28.1.235

[76] S. R. Hall, F. H. Allen, I. D. Brown, The crystallographic information file (CIF): a new standard archive file for crystallography, Acta Crystallographica Section A Foundations of Crystallography 47 (6) (1991) 655–685. doi:10.1107/s010876739101067x.
URL https://journals.iucr.org/paper?S010876739101067X

[77] J. D. Westbrook, P. E. Bourne, STAR/mmCIF: An ontology for macromolecular structure, Bioinformatics 16 (2) (2000) 159–168. doi:10.1093/bioinformatics/16.2.159.
URL https://academic.oup.com/bioinformatics/article/16/2/159/232068

[78] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, S. J. Plimpton, LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Computer Physics Communications 271 (2022) 108171. doi:10.1016/j.cpc.2021.108171.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465521002836

[79] P. Hirel, Atomsk: A tool for manipulating and converting atomic data files, Computer Physics Communications 197 (2015) 212–219. doi:10.1016/j.cpc.2015.07.012.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465515002817

[80] N. M. O'Boyle, M. Banck, C. A. James, C. Morley, T. Vandermeersch, G. R. Hutchison, Open Babel: An open chemical toolbox, Journal of Cheminformatics 3 (1) (2011) 33. doi:10.1186/1758-2946-3-33.
URL https://jcheminf.biomedcentral.com/articles/10.1186/1758-2946-3-33

[81] A. Stukowski, Visualization and analysis of atomistic simulation data with OVITO–the Open Visualization Tool, Modelling and Simulation in Materials Science and Engineering 18 (1) (2009) 015012. doi:10.1088/0965-0393/18/1/015012.
URL https://iopscience.iop.org/article/10.1088/0965-0393/18/1/015012

[82] N. M. O'boyle, A. L. Tenderholt, K. M. Langner, cclib: A library for package-independent computational chemistry algorithms, Journal of Computational Chemistry 29 (5) (2007) 839–845. doi:10.1002/jcc.20823.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.20823

[83] T. Verstraelen, W. Adams, L. Pujal, A. Tehrani, B. D. Kelly, L. Macaya, F. Meng, M. Richer, R. Hernández-Esparza, X. D. Yang, M. Chan, T. D. Kim, M. Cools-Ceuppens, V. Chuiko, E. Vöhringer-Martinez, P. W. Ayers, F. Heidar-Zadeh, IOData: A Python library for reading, writing, and converting computational chemistry file formats and generating input files, Journal of Computational Chemistry 42 (6) (2020) 458–464. doi:10.1002/jcc.26468.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.26468

[84] M. Ceriotti, J. More, D. E. Manolopoulos, i-PI: A Python interface for ab initio path integral molecular dynamics simulations, Computer Physics Communications 185 (3) (2014) 1019–1026. doi:10.1016/j.cpc.2013.10.027.
URL https://linkinghub.elsevier.com/retrieve/pii/S001046551300372X

[85] V. Kapil, M. Rossi, O. Marsalek, R. Petraglia, Y. Litman, T. Spura, B. Cheng, A. Cuzzocrea, R. H. Meißner, D. M. Wilkins, B. A. Helfrecht, P. Juda, S. P. Bienvenue, W. Fang, J. Kessler, I. Poltavsky, S. Vandenbrande, J. Wieme, C. Corminboeuf, T. D. Kühne, D. E. Manolopoulos, T. E. Markland, J. O. Richardson, A. Tkatchenko, G. A. Tribello, V. Van Speybroeck, M. Ceriotti, i-PI 2.0: A universal force engine for advanced molecular simulations, Computer Physics Communications 236 (2019) 214–223. doi:10.1016/j.cpc.2018.09.020.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465518303436

[86] Y. Litman, V. Kapil, Y. M. Y. Feldman, D. Tisi, T. Begušić, K. Fidanyan, G. Fraux, J. Higer, M. Kellner, T. E. Li, E. S. Pós, E. Stocco, G. Trenins, B. Hirshberg, M. Rossi, M. Ceriotti, i-PI 3.0: A flexible and efficient framework for advanced atomistic simulations, The Journal of Chemical Physics 161 (6) (2024) 062504. doi:10.1063/5.0215869.

URL https://pubs.aip.org/jcp/article/161/6/062504/3308034/i-PI-3-0-A-flexible-and-efficient-framework-for

[87] T. A. Barnes, E. Marin-Rimoldi, S. Ellis, T. D. Crawford, The MolSSI Driver Interface Project: A framework for standardized, on-the-fly interoperability between computational molecular sciences codes, Computer Physics Communications 261 (2021) 107688. doi:10.1016/j.cpc.2020.107688.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465520303386

[88] E. Posenitskiy, V. G. Chilkuri, A. Ammar, M. Hapka, K. Pernal, R. Shinde, E. J. L. Borda, C. Filippi, K. Nakano, O. Kohulák, S. Sorella, P. d. O. Castro, W. Jalby, P. L. Ríos, A. Alavi, A. Scemama, TREXIO: A file format and library for quantum chemistry, The Journal of Chemical Physics 158 (17) (2023) 174801. doi:10.1063/5.0148161.
URL https://pubs.aip.org/jcp/article/158/17/174801/2888846/TREXIO-A-file-format-and-library-for-quantum

[89] T. Bray, The JavaScript object notation (JSON) data interchange format, Tech. Rep. 8259, IETF (2017). doi:10.17487/rfc8259.
URL https://www.rfc-editor.org/info/rfc8259

[90] G. Langdale, D. Lemire, Parsing gigabytes of JSON per second, The VLDB Journal 28 (6) (2019) 941–960. doi:10.1007/s00778-019-00578-5.
URL http://link.springer.com/10.1007/s00778-019-00578-5

[91] H. Bekker, Unification of box shapes in molecular simulations, Journal of Computational Chemistry 18 (15) (1997) 1930–1942. doi:10.1002/(sici)1096-987x(19971130)18:15<1930::aid-jcc8>3.0.co;2-p.
URL https://onlinelibrary.wiley.com/doi/10.1002/(SICI)1096-987X(19971130)18:15<1930::AID-JCC8>3.0.CO;2-P

[92] IEEE Computer Society, IEEE standard for floating-point arithmetic (2019). doi:10.1109/ieeestd.2019.8766229.
URL https://ieeexplore.ieee.org/document/8766229/

[93] D. Goldberg, What every computer scientist should know about floating-point arithmetic, ACM Computing Surveys 23 (1) (1991) 5–

48. doi:10.1145/103162.103163.
URL https://dl.acm.org/doi/10.1145/103162.103163

[94] W. D. Clinger, How to read floating point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation – PLDI '90, ACM Press, 1990, pp. 92–101. doi:10.1145/93542.93557.
URL http://portal.acm.org/citation.cfm?doid=93542.93557

[95] W. D. Clinger, How to read floating point numbers accurately, ACM SIGPLAN Notices 39 (4) (2004) 360–371. doi:10.1145/989393.989430.
URL https://dl.acm.org/doi/10.1145/989393.989430

[96] G. L. Steele, J. L. White, How to print floating-point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation – PLDI '90, ACM Press, 1990, pp. 112–126. doi:10.1145/93542.93559.
URL http://portal.acm.org/citation.cfm?doid=93542.93559

[97] G. L. Steele, J. L. White, How to print floating-point numbers accurately, ACM SIGPLAN Notices 39 (4) (2004) 372–389. doi:10.1145/989393.989431.
URL https://dl.acm.org/doi/10.1145/989393.989431

[98] R. G. Burger, R. K. Dybvig, Printing floating-point numbers quickly and accurately, in: Proceedings of the ACM SIGPLAN 1996 conference on Programming language design and implementation, ACM, 1996, pp. 108–116. doi:10.1145/231379.231397.
URL https://dl.acm.org/doi/10.1145/231379.231397

[99] F. Loitsch, Printing floating-point numbers quickly and accurately with integers, in: Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2010, pp. 233–243. doi:10.1145/1806596.1806623.
URL https://dl.acm.org/doi/10.1145/1806596.1806623

[100] M. Andrysco, R. Jhala, S. Lerner, Printing floating-point numbers: a faster, always correct method, in: Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming

Languages, ACM, 2016, pp. 555–567. doi:10.1145/2837614.2837654.
URL https://dl.acm.org/doi/10.1145/2837614.2837654

[101] U. Adams, Ryū: fast float-to-string conversion, in: Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2018, pp. 270–282. doi:10.1145/3192366.3192369.
URL https://dl.acm.org/doi/10.1145/3192366.3192369

[102] P. Deutsch, GZIP file format specification version 4.3, Tech. Rep. 1952, IETF (1996). doi:10.17487/rfc1952.
URL https://www.rfc-editor.org/info/rfc1952

[103] Y. Collet, Zstandard compression and the 'application/zstd' media type, Tech. Rep. 8878, IETF (2021). doi:10.17487/rfc8878.
URL https://www.rfc-editor.org/info/rfc8878

[104] A. L. Spek, checkCIF validation ALERTS: what they mean and how to respond, Acta Crystallographica Section E Crystallographic Communications 76 (1) (2020) 1–11. doi:10.1107/s2056989019016244.
URL https://journals.iucr.org/paper?S2056989019016244

[105] R. Goswami, readcon-core: a CON/convel reader/writer with FFI bindings, Zenodo, software, version 0.17.0 (2026). doi:10.5281/zenodo.23302640.

[106] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. Fernández del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (7825) (2020) 357–362. doi:10.1038/s41586-020-2649-2.
URL https://www.nature.com/articles/s41586-020-2649-2

[107] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, SIAM Review 59 (1) (2017) 65–98. doi:10.1137/141000671.
URL https://epubs.siam.org/doi/10.1137/141000671

[108] M. J. T. Oliveira, N. Papior, Y. Pouillon, V. Blum, E. Artacho, D. Caliste, F. Corsetti, S. d. Gironcoli, A. M. Elena, A. García, V. M. García-Suárez, L. Genovese, W. P. Huhn, G. Huhs, S. Kokott, E. Küçükbenli, A. H. Larsen, A. Lazzaro, I. V. Lebedeva, Y. Li, D. López-Durán, P. López-Tarifa, M. Lüders, M. A. L. Marques, J. Minar, S. Mohr, A. A. Mostofi, A. O'Cais, M. C. Payne, T. Ruh, D. G. A. Smith, J. M. Soler, D. A. Strubbe, N. Tancogne-Dejean, D. Tildesley, M. Torrent, V. W.-z. Yu, The CECAM electronic structure library and the modular software development paradigm, The Journal of Chemical Physics 153 (2) (2020) 024117. doi:10.1063/5.0012901.
URL https://pubs.aip.org/jcp/article/153/2/024117/1061500/The-CECAM-electronic-structure-library-and-the

[109] R. Jung, J.-H. Jourdan, R. Krebbers, D. Dreyer, RustBelt: securing the foundations of the Rust programming language, Proceedings of the ACM on Programming Languages 2 (POPL) (2017) 1–34. doi:10.1145/3158154.
URL https://dl.acm.org/doi/10.1145/3158154

[110] J. M. Perkel, Why scientists are turning to Rust, Nature 588 (7836) (2020) 185–186. doi:10.1038/d41586-020-03382-2.
URL https://www.nature.com/articles/d41586-020-03382-2

[111] D. Lemire, Number parsing at a gigabyte per second, Software: Practice and Experience 51 (8) (2021) 1700–1727. doi:10.1002/spe.2984.
URL https://onlinelibrary.wiley.com/doi/10.1002/spe.2984

[112] N. Mushtak, D. Lemire, Fast number parsing without fallback, Software: Practice and Experience 53 (7) (2023) 1467–1471. doi:10.1002/spe.3198.
URL https://onlinelibrary.wiley.com/doi/10.1002/spe.3198

[113] R. Goswami, readcon-db: mmap CON corpus store with multi-language bindings, https://github.com/lode-org/readcon-db, software, version 0.1.8 (2026).

[114] G. Henkelman, H. Jónsson, Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points, The Journal of Chemical Physics 113 (22) (2000) 9978–9985. doi:10.1063/1.1323224.

URL https://pubs.aip.org/jcp/article/113/22/9978/184858/Improved-tangent-estimate-in-the-nudged-elastic

[115] R. Goswami, Two-dimensional RMSD projections for reaction path visualization and validation, MethodsX 16 (2026) 103851. doi:10.1016/j.mex.2026.103851.
URL https://linkinghub.elsevier.com/retrieve/pii/S2215016126000683

[116] G. L. Kellogg, P. J. Feibelman, Surface self-diffusion on Pt(001) by an atomic exchange mechanism, Physical Review Letters 64 (26) (1990) 3143–3146. doi:10.1103/physrevlett.64.3143.
URL https://link.aps.org/doi/10.1103/PhysRevLett.64.3143

[117] R. Goswami, M. Gunde, H. Jónsson, Enhanced climbing image nudged elastic band method with Hessian eigenmode alignment, Frontiers in Chemistry 14 (2026) 1807063. doi:10.3389/fchem.2026.1807063.
URL https://www.frontiersin.org/articles/10.3389/fchem.2026.1807063/full

[118] T. van Mourik, G. J. Harris, O. L. Polyansky, J. Tennyson, A. G. Császár, P. J. Knowles, Ab initio global potential, dipole, adiabatic, and relativistic correction surfaces for the HCN–HNC system, The Journal of Chemical Physics 115 (8) (2001) 3706–3718. doi:10.1063/1.1383586.
URL https://pubs.aip.org/jcp/article/115/8/3706/463154/Ab-initio-global-potential-dipole-adiabatic-and

[119] R. Goswami, Reproducible orchestration of best practices for reaction path optimization with the nudged elastic band, MethodsX 16 (2026) 103899. doi:10.1016/j.mex.2026.103899.
URL https://linkinghub.elsevier.com/retrieve/pii/S2215016126001172

[120] B. Efron, Better bootstrap confidence intervals, Journal of the American Statistical Association 82 (397) (1987) 171–185. doi:10.1080/01621459.1987.10478410.
URL http://www.tandfonline.com/doi/abs/10.1080/01621459.1987.10478410

[121] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 international symposium on memory management, ACM, 2013, pp. 63–74. doi:10.1145/2491894.2464160.
URL https://dl.acm.org/doi/10.1145/2491894.2464160

[122] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th international conference on Architectural support for programming languages and operating systems, ACM, 2009, pp. 265–276. doi:10.1145/1508244.1508275.
URL https://dl.acm.org/doi/10.1145/1508244.1508275

[123] T. Hoefler, R. Belli, Scientific benchmarking of parallel computing systems, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, ACM, 2015, pp. 1–12. doi:10.1145/2807591.2807644.
URL https://dl.acm.org/doi/10.1145/2807591.2807644

[124] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[125] E. Jakupovic, O. Beckstein, MPI-parallel molecular dynamics trajectory analysis with the H5MD format in the MDAnalysis Python package, in: Proceedings of the Python in Science Conference, SciPy, 2021, pp. 40–48. doi:10.25080/majora-1b6fd038-005.
URL https://doi.curvenote.com/10.25080/majora-1b6fd038-005

[126] R. Gowers, O. Beckstein, M. Linke, I. Alibay, J. Barnoud, T. Reddy, H. MacDermott-Opeskin, R. Meli, L. Wang, U. Bansal, J. Zeman, J. Detlefs, D. L. Dotson, M. N. Melo, M. Bieniek, ayushsuhane, S. Buchoux, M. Tiberti, N. Michaud-Agrawal, D. Cruz, S. Seyler, denniej0 2, P. Loche, J. Borreguero, Y. Zhuang, T. Yelgoe, N. Pal, Ninad, J. Domański, Z. Wu, MDAnalysis/mdanalysis: Release 2.10.0 of MDAnalysis and MDAnalysisTests, Zenodo, software, version package-2.10.0 (2025). doi:10.5281/zenodo.17382469.

[127] A. Collette, T. Kluyver, T. A. Caswell, J. Tocknell, J. Kieffer, A. Jelenak, A. Scopatz, D. Dale, Chen, T. VINCENT, M. Einhorn, Payno, Juliagarriga, P. Sciarelli, V. Valls, S. Ghosh, U. K. Pedersen, Jakirkham,

M. Raspaud, C. Danilevski, H. Abbasi, J. Readey, K. Mühlbauer, A. Paramonov, L. Chan, V. A. Solé, Jialin, D. H. Guest, Y. Feng, M. Kittisopikul, h5py/h5py: 3.7.0, Zenodo, software, version 3.7.0 (2022). doi:10.5281/zenodo.6575970.

[128] R. Goswami, readcon-core: a CON/convel reader/writer with FFI bindings, Zenodo, software, all versions (2026). doi:10.5281/zenodo.18753567.

[129] A. M. Smith, D. S. Katz, K. E. Niemeyer, F. S. C. W. Group, Software citation principles, PeerJ Computer Science 2 (2016) e86. doi:10.7717/peerj-cs.86.
URL https://peerj.com/articles/cs-86

[130] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR Guiding Principles for scientific data management and stewardship, Scientific Data 3 (1) (2016) 160018. doi:10.1038/sdata.2016.18.
URL https://www.nature.com/articles/sdata201618

[131] M. Barker, N. P. Chue Hong, D. S. Katz, A.-L. Lamprecht, C. Martinez-Ortiz, F. Psomopoulos, J. Harrow, L. J. Castro, M. Gruenpeter, P. A. Martinez, T. Honeyman, Introducing the FAIR Principles for research software, Scientific Data 9 (1) (2022) 622. doi:10.1038/s41597-022-01710-x.
URL https://www.nature.com/articles/s41597-022-01710-x

[132] A.-L. Lamprecht, L. Garcia, M. Kuzak, C. Martinez, R. Arcila, E. M. D. Pico, V. D. D. Angel, S. v. d. Sandt, J. Ison, P. A. Martinez, P. McQuilton, A. Valencia, J. Harrow, F. Psomopoulos, J. L. Gelpi, N. C. Hong,

C. Goble, S. Capella-Gutierrez, Towards FAIR principles for research software, Data Science 3 (1) (2019) 37–59. doi:10.3233/ds-190026.
URL https://journals.sagepub.com/doi/10.3233/DS-190026

[133] C. W. Andersen, R. Armiento, E. Blokhin, G. J. Conduit, S. Dwaraknath, M. L. Evans, Á. Fekete, A. Gopakumar, S. Gražulis, A. Merkys, F. Mohamed, C. Oses, G. Pizzi, G.-M. Rignanese, M. Scheidgen, L. Talirz, C. Toher, D. Winston, R. Aversa, K. Choudhary, P. Colinet, S. Curtarolo, D. Di Stefano, C. Draxl, S. Er, M. Esters, M. Fornari, M. Giantomassi, M. Govoni, G. Hautier, V. Hegde, M. K. Horton, P. Huck, G. Huhs, J. Hummelshøj, A. Kariryaa, B. Kozinsky, S. Kumbhar, M. Liu, N. Marzari, A. J. Morris, A. A. Mostofi, K. A. Persson, G. Petretto, T. Purcell, F. Ricci, F. Rose, M. Scheffler, D. Speckhard, M. Uhrin, A. Vaitkus, P. Villars, D. Waroquiers, C. Wolverton, M. Wu, X. Yang, OPTIMADE, an API for exchanging materials data, Scientific Data 8 (1) (2021) 217. doi:10.1038/s41597-021-00974-z.
URL https://www.nature.com/articles/s41597-021-00974-z

[134] C. Draxl, M. Scheffler, The NOMAD laboratory: from data sharing to artificial intelligence, Journal of Physics: Materials 2 (3) (2019) 036001. doi:10.1088/2515-7639/ab13bb.
URL https://iopscience.iop.org/article/10.1088/2515-7639/ab13bb

[135] K. T. Winther, M. J. Hoffmann, J. R. Boes, O. Mamun, M. Bajdich, T. Bligaard, Catalysis-Hub.org, an open electronic structure database for surface reactions, Scientific Data 6 (1) (2019) 75. doi:10.1038/s41597-019-0081-y.
URL https://www.nature.com/articles/s41597-019-0081-y

[136] M. Schreiner, A. Bhowmik, T. Vegge, J. Busk, O. Winther, Transition1x – a dataset for building generalizable reactive machine learning potentials, Scientific Data 9 (1) (2022) 779. doi:10.1038/s41597-022-01870-w.
URL https://www.nature.com/articles/s41597-022-01870-w

[137] L. Chanussot, A. Das, S. Goyal, T. Lavril, M. Shuaibi, M. Riviere, K. Tran, J. Heras-Domingo, C. Ho, W. Hu, A. Palizhati, A. Sriram, B. Wood, J. Yoon, D. Parikh, C. L. Zitnick, Z. Ulissi, Open catalyst

2020 (OC20) dataset and community challenges, ACS Catalysis 11 (10) (2021) 6059–6072. doi:10.1021/acscatal.0c04525.
URL https://pubs.acs.org/doi/10.1021/acscatal.0c04525